Assignment 8 - Optimize Anything

Optimization Techniques - Technical report. All numbers below are copied from the output of the scripts in this repository (folder *results/*); settings and seeds are fixed in the code. Code was written with AI assistance (Claude); see the AI tools register in the README.

1. Main case study (Track E): delivery routing

Process: one vehicle must visit 60 delivery stops in a 100 x 100 km area and return to the depot. **Baseline:** visiting stops in the order they arrive (random order, mean of 20). **Formulation:** decision variable = order of the stops (a permutation); objective = minimise total tour length in km; constraint = every stop visited exactly once. **Assumptions:** straight-line distances, one vehicle, no time windows, synthetic data (seed 0).

Variable type	Problem size	Exact or heuristic	Hardware	Parallelism	Chosen methods
Discrete (permutation)	60 stops (60! tours)	Exact only feasible for about 12-15 stops (Held-Karp DP); heuristics for 60	1 CPU core	Not needed (runs in ms to seconds)	Nearest neighbour, 2-opt, simulated annealing; Held-Karp for checking

```
== E1. Small instance (12 stops): exact optimum vs heuristics ==
Held-Karp (exact DP): 341.33  (0.04s)
nearest neighbour: 359.45   NN+2-opt: 344.24

== E2. Main instance: 60 delivery stops in a 100x100 km area (distance in km) ==
baseline (visit in given/random order, mean of 20): 3231.7 km
nearest neighbour: 741.2 km  (0.3 ms)  improvement over baseline 77.1%
NN + 2-opt:        662.0 km  (5.1 ms)  improvement over baseline 79.5%
simulated annealing (5 seeds): mean 657.8 +- 17.9 km  (0.5 s per run)
correctness check passed: every stop visited exactly once
```

Result: nearest neighbour is about 77% shorter than the baseline, 2-opt about 80%, and simulated annealing is similar to 2-opt but 100x slower and varies between seeds. On the 12-stop check, 2-opt is within 1% of the exact optimum. Limitations: no time windows or capacities; for larger real problems use ILP/OR-tools or parallel heuristics.

2. Track A - Continuous optimization

```
== A1. Optimizers on convex and non-convex objectives (200 iters, lr=0.05) ==
-- convex start=[4.0, 1.0]
GD        final f=0.00000 x=[0. 0.] time=0.41 ms
Momentum  final f=0.00000 x=[ 0. -0.] time=0.64 ms
NAG       final f=0.00000 x=[0. 0.] time=1.37 ms
AdaGrad   final f=0.00000 x=[0. 0.] time=0.83 ms
RMSProp   final f=0.03381 x=[0.257 0.009] time=1.09 ms
Adam      final f=0.00000 x=[0.003 0.   ] time=1.69 ms
-- non-convex start=[1.5, 1.0]
GD        final f=-0.06128 x=[0.879 0.   ] time=0.48 ms
Momentum  final f=-0.45916 x=[-1.088  0.   ] time=0.70 ms
NAG       final f=-0.06128 x=[0.879 0.   ] time=0.89 ms
AdaGrad   final f=-0.06128 x=[0.879 0.   ] time=0.95 ms
RMSProp   final f=-0.06118 x=[0.869 0.009] time=1.13 ms
Adam      final f=-0.06128 x=[0.879 0.   ] time=1.75 ms

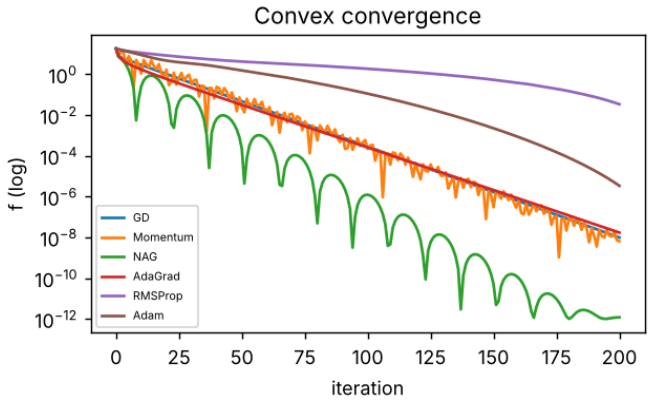
== A2. Newton: convex converges in 1 step; non-convex converges to a saddle/min depending on start ==
convex  from [4,1]  -> [0. 0.]
non-cvx from [1.5, 1.0] -> [0.879 0.   ] f=-0.0613 Hessian eig x-dir=1.317 (min)
non-cvx from [0.3, 1.0] -> [0.209 0.   ] f=0.0204 Hessian eig x-dir=-0.869 (max/saddle)
non-cvx from [0.05, 1.0] -> [0.209 0.   ] f=0.0204 Hessian eig x-dir=-0.869 (max/saddle)
non-cvx from [-1.5, 1.0] -> [-1.088  0.   ] f=-0.4592 Hessian eig x-dir=2.551 (min)

== A3. Gauss-Newton: fit y = a*exp(b*x) ==
Gauss-Newton estimate (a,b) = [1.994 1.502]  true=(2.0,1.5)

== A4. Backpropagation: 2-layer net on moons, plus gradient check ==
gradient check dL/dW1[0,0]: backprop=0.028022  finite-diff=0.028022
GD        loss=0.3361 acc=0.868 time=0.01s
SGD       loss=0.2214 acc=0.927 time=0.16s
MiniBatch loss=0.2992 acc=0.895 time=0.03s
Adam      loss=0.1894 acc=0.955 time=0.04s

== A5. Curvature tools: Hessian-vector product and diagonal Fisher ==
HVP (finite diff of gradients) = [ 1. 40.]  exact H@v = [ 1. 40.]
diagonal Fisher (output layer, mean g^2) first 4: [0.0059 0.0025 0.0328 0.0126]

== A6. Noisy objective: constant vs decaying step size ==
constant lr=0.1          mean final f=63.1295 +- 79.7153
decay lr=0.5/(1+0.05t)   mean final f=0.0260 +- 0.0170
```



Reading: on the convex bowl every method converges and Newton needs one step. On the non-convex function the result depends on the start: Newton can land on the saddle or a local minimum, and the first-order methods end in the minimum on their side. A decaying step size removes most of the noise in the noisy-gradient test. Backprop was checked against finite differences.

3. Track B - Discrete optimization

```
== B1. 0/1 knapsack (feature/project selection), n=18, capacity=150 ==
Greedy (ratio)         value=661  time=1.26 ms
Dynamic programming    value=690  time=0.77 ms
ILP (scipy milp)       value=690  time=22.93 ms
Branch-and-Bound       value=690  time=0.21 ms
Brute force (check)    value=690  time=982.52 ms
B&B nodes explored: 73 of 524287 possible

== B2. Max-Cut as QUBO / Ising ==
QUBO optimum energy=-10.0  -> cut=10.0; direct brute-force cut=10  (match: True)
Ising energy of the optimum = -10.0 (should equal QUBO energy -10.0)

== B3. Feature selection as QUBO with a penalty for 'choose exactly k' ==
selected features [2, 5, 6, 7], count=4 (constraint k=4 satisfied: True)

== B4. Classical baseline for Track C: simulated annealing on the Max-Cut QUBO ==
SA best cut over 10 seeds: mean=10.00 min=10.0 max=10.0 (optimum 10)
```

Reading: greedy is fast but 4% worse than optimal here; DP, ILP and Branch-and-Bound all give the optimum, and B&B explores very few nodes. The QUBO and Ising energies match the direct Max-Cut value. For the constraint 'choose exactly k features' the penalty term works (k = 4 selected). Penalty weight must be large enough or constraints are violated.

4. Track C - Quantum optimization (simulator, not hardware)

```
== C1. QAOA on the Track B Max-Cut graph (exact simulation, not hardware) ==
optimal cut = 10, random-guess expected cut = 6.00
p=1: expected cut=8.309  approximation ratio=0.831
p=2: expected cut=9.248  approximation ratio=0.925
p=3: expected cut=9.654  approximation ratio=0.965

== C2. Finite shots and a simple noise model on the p=3 QAOA ==
shots=    10  estimate=9.617 +- 0.272  (exact 9.654)
shots=   100  estimate=9.679 +- 0.057  (exact 9.654)
shots=  1000  estimate=9.654 +- 0.022  (exact 9.654)
shots= 10000  estimate=9.654 +- 0.007  (exact 9.654)
gate error 0.000 -> expected cut 9.654
gate error 0.001 -> expected cut 9.441
gate error 0.005 -> expected cut 8.705
gate error 0.020 -> expected cut 7.087

== C3. VQE (2 qubits). exact ground energy = -1.4142 ==
parameter-shift GD         final energy -1.4141 +- 0.0001  (8 circuit evals/step)
SPSA                       final energy -1.4022 +- 0.0228  (2 circuit evals/step)
quantum natural gradient   final energy -1.4142 +- 0.0000  (8 circuit evals/step)

== C4. Barren plateau: variance of a parameter-shift gradient, global cost, random RY+CZ circuit ==
qubits= 2  Var[grad]=5.83e-02
qubits= 4  Var[grad]=5.67e-03
qubits= 6  Var[grad]=4.85e-04
qubits= 8  Var[grad]=6.67e-05
qubits=10  Var[grad]=9.11e-06
```

Reading: QAOA improves with depth p (0.83 to 0.97 approximation ratio). Few shots give noisy estimates (the error shrinks as 1/sqrt(shots)); a simple noise model lowers the result quickly with gate error; and gradient variance drops about 10x for every 2 extra qubits (barren plateau). All results are simulated with a noise-free state-vector simulator plus the simple noise formula shown; no real hardware was used.

5. Track D - Submodular, parallel and distributed

```
== D1. Greedy on a submodular objective (coverage) ==
diminishing-returns violations in 2000 random tests: 0 (0 => submodular, so the 1-1/e guarantee applies)
greedy covers 39, optimum 39, ratio 1.000 >= 1-1/e = 0.632

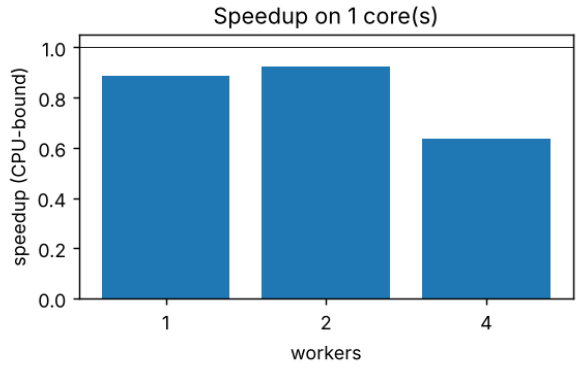
== D2. Parallel speedup (machine has 1 CPU core(s)) ==
CPU-bound, serial: 0.283s
CPU-bound workers=1: 0.319s speedup=0.89 efficiency=0.89
CPU-bound workers=2: 0.307s speedup=0.92 efficiency=0.46
CPU-bound workers=4: 0.444s speedup=0.64 efficiency=0.16
waiting-bound (sleep) workers=1: 1.218s speedup=0.99 efficiency=0.99
waiting-bound (sleep) workers=2: 0.617s speedup=1.95 efficiency=0.97
waiting-bound (sleep) workers=4: 0.326s speedup=3.68 efficiency=0.92
waiting-bound (sleep) workers=8: 0.181s speedup=6.62 efficiency=0.83
(CPU-bound speedup is limited by the number of physical cores; waiting-bound tasks overlap even on one core.)

Load balancing: uneven tasks, static chunks vs dynamic (work-stealing-like) scheduling, 4 workers
static chunks of 10: 0.309s   dynamic (chunk=1): 0.217s   ideal makespan ~0.200s

Pipeline parallelism (2 stages = 2 'layers' on separate processes, which is also the simplest model-parallel split):
    sequential 0.80s vs pipeline 0.43s, speedup 1.87 (ideal 2.0)
Amdahl: with serial fraction f the speedup on P workers is 1/(f+(1-f)/P); f=0.1 -> P=4: 3.08, P=inf: 10

== D3. FedAvg vs centralized (logistic regression, 5 clients, non-IID) ==
centralized GD (200 steps): loss=0.2406
FedAvg local_steps= 1 rounds= 5: loss=0.6004  (communication = 5 rounds)
FedAvg local_steps= 1 rounds=20: loss=0.4677  (communication = 20 rounds)
FedAvg local_steps= 5 rounds= 5: loss=0.4431  (communication = 5 rounds)
FedAvg local_steps= 5 rounds=20: loss=0.2929  (communication = 20 rounds)
FedAvg local_steps=20 rounds= 5: loss=0.2939  (communication = 5 rounds)
FedAvg local_steps=20 rounds=20: loss=0.2055  (communication = 20 rounds)

== D4. Two-task shared-parameter model: equal weights vs MGDA vs GradNorm ==
equal     L1=7.6676 L2=0.0767 sum=7.7443
MGDA      L1=1.0366 L2=415.3545 sum=416.3911
GradNorm  L1=6.4928 L2=6.1821 sum=12.6749
```



Reading: the machine used has **1 CPU core**, so CPU-bound tasks show no speedup (an honest negative result: more processes only add overhead). Tasks that wait (sleep) overlap well, as does the 2-stage pipeline, and dynamic scheduling fixes the imbalance of uneven tasks. Greedy coverage is submodular (0 violations found) and reached the optimum. FedAvg approaches centralized training as local steps and rounds increase, at the cost of communication. In the two-task test, equal weights and GradNorm gave balanced results while MGDA drove task 1 down but left task 2 (100x larger scale) high after 300 steps; this was not tuned, so no general claim is made.

6. Track F - Compiler optimization

```
== F1. Unoptimized program ==
IR instructions=27  est. cycles=41  output=[39, 144, 32]

== F2. Each pass alone, then pass-ordering as an optimization problem ==
const     alone: instrs=27 cycles= 35 output_ok=True
strength  alone: instrs=27 cycles= 35 output_ok=True
cse       alone: instrs=27 cycles= 41 output_ok=True
dce       alone: instrs=24 cycles= 36 output_ok=True
exhaustive search over 24 orders: best ('cse', 'const', 'strength', 'dce') -> cycles=13 instrs=9; worst ('strength',
    'cse', 'dce', 'const') -> cycles=28
greedy (apply whichever pass helps most next): ['const', 'dce', 'strength', 'cse'] -> cycles=15

== F3. Before/after measurement ==
                instrs  cycles   run time (20k runs)
unoptimized         27      41              0.126 s
optimized            9      13              0.046 s
extra compile time for the passes: 0.08 ms;  same output: True
correctness: optimized and unoptimized programs give identical output on 4 different inputs

optimized IR:
shl t3 x 1
add t4 x y
mul t6 t4 t4
mul t8 t4 3
add t11 t3 t8
sub t14 t11 11
print None t14 None
print None t6 None
print None 32 None
```

Reading: four passes (constant folding/propagation, strength reduction, common sub-expression elimination, dead-code elimination) reduced the estimated cycles from 41 to 13. Pass order matters: the best of 24 orders gives 13 cycles, the worst 28, and the greedy order gives 15. Output was identical on every tested input.

7. Track G - xv6 operating system

Change: priority scheduler (lower number = higher priority, ties by least recently run), new system call *setpriority(pid, prio)* and a benchmark program *schedbench* with 3 workloads. **Objective:** turnaround time of important jobs. The patch is in *track_g_xv6_priority_scheduler.patch*. **Status: the patch was NOT compiled, booted under QEMU or run through usertests in this environment (no RISC-V toolchain).** The table below is a Python **simulation** of the same three workloads, not an xv6 measurement. Real before/after numbers must be added after building the patch.

```
== G1. Scheduler simulation (time units = ticks). Turnaround of every job, high-priority jobs listed first ==
W1 CPU-bound, 2 high + 2 low priority
   round robin  turnarounds=[797, 798, 799, 800]  mean=798.5  mean of highest-priority jobs=797.5  makespan=800
   priority     turnarounds=[399, 400, 799, 800]  mean=599.5  mean of highest-priority jobs=399.5  makespan=800
W2 twelve short equal jobs
   round robin  turnarounds=[229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240]  mean=234.5  mean of highest-
    priority jobs=234.5  makespan=240
   priority     turnarounds=[229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240]  mean=234.5  mean of highest-
    priority jobs=234.5  makespan=240
W3 CPU + sleepy mix
   round robin  turnarounds=[571, 603, 573, 604]  mean=587.8  mean of highest-priority jobs=587.0  makespan=604
   priority     turnarounds=[259, 320, 560, 616]  mean=438.8  mean of highest-priority jobs=289.5  makespan=616
Reading: priority scheduling cuts turnaround for the important jobs (W1, W3) at the cost of the low-priority ones;
    with equal priorities (W2) it behaves like round robin.
```

8. Track H - DBMS (SQLite)

```
== H1. Synthetic dataset: orders table, 100000 rows (documented, seed 0) ==
baseline workload time (weighted by frequency) = 545.1 ms, db size = 2832 KB
baseline plans:
    SCAN orders
    SCAN orders
    SCAN orders | USE TEMP B-TREE FOR GROUP BY
    SCAN orders
    SCAN orders

== H2. Measure benefit and size of each candidate index ==
cust               size=   1048 KB  time saved= 298.71 ms
product_status     size=   1192 KB  time saved= -13.19 ms
day                size=   1012 KB  time saved=-205.21 ms
cust_day           size=   1312 KB  time saved= 259.76 ms
region_status_day  size=   1344 KB  time saved=-619.73 ms
status             size=    904 KB  time saved= -65.64 ms
region             size=    932 KB  time saved=-424.09 ms
product            size=   1036 KB  time saved= -49.84 ms

storage budget = 1500 KB

== H3. Greedy vs DP vs ILP vs default (no index) ==
greedy (benefit/size)  chose ['cust']
                       workload 249.27 ms (baseline 545.1 ms, speedup 2.2x), extra storage 1048 KB, results identical:
    True
dynamic programming    chose ['cust']
                       workload 254.58 ms (baseline 545.1 ms, speedup 2.1x), extra storage 1048 KB, results identical:
    True
  plans with DP choice:
      SEARCH orders USING INDEX ix_cust (cust=?)
      SCAN orders
      SCAN orders | USE TEMP B-TREE FOR GROUP BY
      SEARCH orders USING INDEX ix_cust (cust=?)
      SCAN orders
ILP (milp)             chose ['cust']
                       workload 399.79 ms (baseline 545.1 ms, speedup 1.4x), extra storage 1048 KB, results identical:
    True
(Benefits are measured one index at a time and assumed additive; the combined measurement above checks that
    assumption.)
```

Formulation: choose a subset of 8 candidate indexes to maximise measured workload time saved, subject to total index size at most 1500 KB (a knapsack). Greedy, DP and ILP were compared with the default (no index) plan. Query results were identical (hash check) and EXPLAIN QUERY PLAN shows two queries switching from a table scan to an index search. Benefits are measured one index at a time and assumed additive. Several indexes made the workload slower (the default planner chose them badly), which is why measured benefit, not intuition, is used. Timing noise exists: the three methods chose the same index but reported 1.4x to 2.2x.

9. Track I - Quantum algorithms and QML

```
== I1. Deutsch-Jozsa (3 input qubits) ==
constant              P(input=000) = 1.000 -> constant
balanced (x0 xor x1)  P(input=000) = 0.000 -> balanced

== I2. Bernstein-Vazirani, secret 1011 ==
measured input register: 1011

== I3. Grover search vs classical linear search (oracle queries) ==
qubits       N  Grover queries  P(success)  classical avg  sim time
     4      16               3      0.9613              8   0.0023s
     6      64               6      0.9966             32   0.0003s
     8     256              12      0.9999            128   0.0003s
    10    1024              25      0.9995            512   0.0006s
    12    4096              50      0.9999           2048   0.0014s
    14   16384             100      1.0000           8192   0.0051s
Query count shows the quadratic advantage (theoretical). Wall-clock on a classical simulator is NOT faster: the
    simulator pays 2^n work per query.

== I4. QFT check and quantum phase estimation ==
QFT circuit matches matrix: True
QPE: true phase 0.15625, estimated 5/32 = 0.15625

== I5. Shor's algorithm, N=15, a=7 (order finding with QPE, then classical post-processing) ==
measurement 64/256 -> order r=4 -> factors 3 x 5

== I6. Quantum annealing: not simulated physically. Simulated annealing (Track B4) is used as the classical stand-in.
Limit: real annealers need the QUBO embedded on a sparse hardware graph; dense problems need chains of physical
    qubits.

== I7. QML vs classical on moons data (2 qubits, simulator) ==
model                        test acc  params  train time  circuit evals
variational classifier (VQC)     0.55       8       1.23s           4800
quantum kernel + SVM             0.75       -       0.53s    80*80 pairs
logistic regression              0.78       3     0.0074s              0
RBF SVM                          0.80       -      <0.01s              0
Honest reading: on this small dataset the classical models are as good or better and far cheaper; the quantum models
    run on a classical simulator, so no quantum speedup is claimed.
```

```
Hardware: no real quantum hardware was used; a noise model is in Track C (C2).
```

Reading: all circuits (Deutsch-Jozsa, Bernstein-Vazirani, Grover, QFT, QPE, Shor for N = 15) were written by hand on our own simulator and gave the correct answers. Grover needs about sqrt(N) queries versus N/2, which is a theoretical, query-count advantage; the simulator itself gets no wall-clock speedup. The small quantum classifiers were not better than classical models (honest negative result). Compare with Track D, where parallel speedups can be measured directly on real hardware when cores exist.

10. Track J - Software engineering: test-suite minimisation

```
== J1. Synthetic suite: 150 tests, 500 lines, total runtime 404 s ==
Formulation: min sum(cost_t * x_t)  s.t. for every line l: sum(x_t for tests covering l) >= 1, x_t in {0,1}
method                  tests  runtime(s)  coverage kept  solve time
naive: run all            150       404.1            yes           -
greedy                     87       178.9           True      0.004s
ILP (exact)                72       163.0           True      1.694s
Cost of minimisation: coverage of lines is preserved, but fault detection is NOT guaranteed (a dropped test may catch
    a bug that line coverage cannot see).
Limitation: data is synthetic; for the 1000+ line codebase requirement, replace `cov` with real coverage data (e.g.
    from pytest --cov) of a project.
```

Reading: greedy set cover keeps all line coverage with 58% of the tests and 44% of the run time; the ILP is slightly better (48% of tests, 40% of run time) but slower to solve. Trade-off: removing tests can lose bug-detection power. Data are synthetic.

11. Chapter 6 coverage checklist

Chapter 6 item	Where	Status
ML as optimization, convex vs non-convex, saddle/local minima	Track A (A1, A2)	Done
GD, SGD, mini-batch, Momentum, NAG, AdaGrad, RMSProp, Adam	Track A (A1, A4)	Done
Newton, Gauss-Newton, HVP, Fisher, noisy objective, backprop	Track A (A2-A6)	Done (Fisher: diagonal only)
Greedy, ILP, Branch-and-Bound, DP, QUBO/Ising, penalty terms	Track B	Done
QAOA, VQE, parameter-shift, SPSA, QNG, shots, noise, barren plateau	Track C	Done on simulator
Deutsch-Jozsa, BV, Grover, QFT, QPE, Shor, VQC, quantum kernel	Track I	Done on simulator
Quantum annealing	Track I (I6)	Simulated annealing stand-in only
Submodular greedy and its conditions	Track D (D1)	Done
Load balancing, work stealing, speedup, efficiency, Amdahl/Gustafson	Track D (D2)	Dynamic scheduling, pipeline, Amdahl shown; Gustafson only mentioned; 1-core machine
Data / model / pipeline parallelism	Track D (D2)	Data and pipeline; model parallel = 2-stage pipeline
FedAvg, MGDA, GradNorm	Track D (D3, D4)	Done (GradNorm simplified)
Strategy selection	Section 1 table	Done

12. Final questions (short answers)

1. Process / evidence: Delivery routing; a random-order tour is about 3200 km vs about 660 km after optimization.

2. Formulation: Section 1 and the formulation line at the top of each track's section.

3. Chapter 6 methods: See the checklist; each was applied to a problem that fits its assumptions.

4. Optimized vs baseline: Routing -80% distance; compiler 41 to 13 cycles; DB workload about 2x faster; test suite 56-60% less run time; priority scheduling halves turnaround of important jobs in simulation.

5. Trade-offs / failures: Greedy loses quality; penalties need tuning; some indexes hurt; test minimisation may lose fault detection; quantum results are simulated and not faster; the CPU-bound parallel test showed no speedup on 1 core; the xv6 patch is untested.

6. Correctness / reproducibility: Fixed seeds, brute-force checks (knapsack, Max-Cut, VQE exact energy), gradient check, identical program output, identical query results, one command (run_all.sh).

7. If scale changes: Larger routing: OR-tools / ILP / parallel heuristics; larger quantum: real hardware or tensor-network simulators; larger DB: more candidates and interaction-aware selection.

8. Compiler, xv6, DBMS: Passes + pass ordering (output checked); priority scheduler (concept: scheduling and context switching; verification still to do on QEMU); index selection (results hashed and plans compared).

9. Quantum vs classical vs parallel: Quantum: provable query advantage (Grover) but simulation overhead, noise, barren plateaus and no QML win here; parallel: real speedup only with real cores or waiting tasks.

10. AI tools: Claude (see README register); errors found are listed in FAILURE_LOG.md. A second-tool comparison still has to be filled in by the student.